■ Face Detection Attendance System

Project Notes & Roadmap — Fareed's Personal Reference

Generated: 03 April 2026

This document summarises the complete project discussion, current architecture, Python package plan, action-item checklist, and all references — written in plain language so you can pick up exactly where you left off at any time.

---

1. Project Overview

Yeh ek **AI-powered attendance system** hai jo classroom ki photo leti hai aur automatically pehchaan leta hai kaun present hai — bina manually roll-call kiye. Teacher sirf ek image upload karta hai, baaki sab system karta hai.

Kya karta hai	Classroom photo se faces detect karo, CSV/database se match karo, attendance ma
Tech Stack	Python · Flask · DeepFace (Facenet) · TensorFlow · OpenCV · scikit-learn · Next.js (
Architecture	Microservice style: Python/Flask backend + Next.js frontend, dono Docker containers
Current Status	Fully working end-to-end system. REST API + frontend + email notification sab live h

2. System Architecture — Step by Step

2.1 Data Flow

Teacher → **uploads 1–6 classroom images** via Next.js frontend → images reach Flask /upload API → images split into 3×3 and 4×4 grids (*AdaptiveGrid strategy*) → each grid cell processed by DeepFace (Facenet model) → cosine similarity against stored encodings → recognized faces returned as JSON → email sent to parents of absent students.

2.2 Core Modules

api/app.py	Flask REST API. Handles image upload, calls recognition, sends emails.
deepface_model/main.py	Main recognition engine. Uses DeepFace + Facenet + cosine similarity.
metrics/division.py	Splits classroom image into 3×3 and 4×4 grid tiles.
api/email_sending.py	Sends attendance emails to parents via SMTP.
recognition/training.py	Trains face encodings from CSV of student images, saves .pkl file.
recognition/face_cache.py	Caches face encodings so training doesn't repeat every run.
research/	Research framework: AdaptiveGrid paper, metrics, visualizer, experiments.
deepface_model/testing.py	Testing utilities for model evaluation.

2.3 AdaptiveGrid Strategy (Research Contribution)

Classroom images mein multiple faces hoti hain, aur ek hi badi image pe face detection accuracy kam hoti hai. Solution: image ko **3×3 aur 4×4 grids** mein tod do, phir har tile pe detection karo. Yeh accuracy significantly improve karta hai. Yeh research component model/research/ folder mein documented hai.

3. Kya Discuss Hua — Summary

3.1 Python Package Idea ■

Discussion: **Is poore system ko ek installable Python package banana** — taaki koi bhi pip install faceattend karke apne school/college mein use kar sake. Yeh idea excellent hai kyunki:

- • Codebase already modular hai — har module alag folder mein hai
- • MIT License already present hai (model/LICENSE)
- • DeepFace, Flask, reportlab sab standard libraries hain
- • Research component bhi packaged ho sakta hai
- • PyPI pe publish hone ke baad globally accessible ho jaayega

3.2 Package ka Naam (Suggestions)

faceattend	Short, memorable, directly describes purpose — best choice
face-attend	PyPI-friendly hyphenated version
classattend	Broader scope — not just faces
deepattend	Highlights DeepFace usage

3.3 Open-Source Fayde

- • ■ Global reach — schools worldwide use kar sakti hain
- • ■ GitHub stars — ML/CV community mein popular ho sakta hai
- • ■ Research paper + package = academic credibility
- • ■ Resume pe strong addition — 'Published Python package on PyPI'
- • ■ Community contributions — features aur bug fixes free mein milenge
- • ■ Potential citations agar researchers use karein

4. Python Package Banana — Complete Plan

4.1 Proposed Package Structure

```
faceattend/
■■■ faceattend/
■ ■■■ __init__.py ← public API (expose main functions)
■ ■■■ recognition/ ← existing deepface_model/ → rename/merge
■ ■ ■■■ __init__.py
■ ■ ■■■ engine.py ← recognize_faces_in_directory()
■ ■ ■■■ training.py ← train_faces()
■ ■■■ grid/
■ ■ ■■■ __init__.py
■ ■ ■■■ strategy.py ← split_image() AdaptiveGrid
■ ■■■ api/
■ ■ ■■■ __init__.py
```

```
■ ■ ■■■ server.py ← Flask app (existing api/app.py)
■ ■ ■■■ email_sending.py ← existing
■ ■■■ research/ ← existing research/ module
■ ■■■ cli.py ← NEW: command-line interface
■■■ tests/ ← existing test/ → restructure
■■■ pyproject.toml ← NEW: package metadata
■■■ README.md ← update with install + quickstart
■■■ CHANGELOG.md ← NEW: version history
■■■ LICENSE ← existing MIT license
```

4.2 pyproject.toml (Key Content)

```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.backends.legacy:build"

[project]
name = "faceattend"
version = "0.1.0"
description = "AI-powered face recognition attendance system"
authors = [{name = "Fareed", email = "your@email.com"}]
license = {text = "MIT"}
requires-python = ">=3.9"
dependencies = ["deepface", "flask", "flask-cors", "opencv-python",
"scikit-learn", "pandas", "numpy", "python-dotenv"]

[project.scripts]
faceattend = "faceattend.cli:main"
```

4.3 CLI Interface Design

```
# Training
faceattend train --csv Data/dataset.csv --output encodings.pkl

# Recognition (single image or directory)
faceattend recognize --image classroom.jpg
faceattend recognize --dir ./images --threshold 0.7

# Start API server
faceattend serve --port 8000 --host 0.0.0.0

# Run research experiments
faceattend research --config experiments.yaml
```

4.4 Public Python API (faceattend/__init__.py)

```
from faceattend.recognition.engine import recognize_faces,
recognize_faces_in_directory
from faceattend.recognition.training import train_faces
from faceattend.grid.strategy import split_image
from faceattend.api.server import create_app

__version__ = "0.1.0"
```

```
__all__ = ["recognize_faces", "recognize_faces_in_directory",
"train_faces", "split_image", "create_app"]
```

5. Kya Karna Hai — Action Checklist ■

Phase 1: Packaging Setup (1–2 din)

■ **pyproject.toml**	Package metadata, dependencies, entry points banana
■ **Directory restructure**	model/ ke andar modules ko faceattend/ package structure mein reorganize karo
■ **__init__.py update**	Clean public API expose karo
■ **cli.py banana**	fire ya argparse se CLI commands add karo
■ **.env config**	Hardcoded paths hata do, sab .env se load karo (dotenv already used hai)

Phase 2: Quality & Tests (2–3 din)

■ **Tests restructure**	model/test/ ko proper pytest structure mein lao
■ **Unit tests add**	train_faces(), recognize_faces(), split_image() ke liye tests
■ **GitHub Actions CI**	.github/workflows/ci.yml banana — auto lint + test on PR
■ **Auto PyPI publish**	.github/workflows/publish.yml — git tag pe auto publish
■ **Code cleanup**	Unused imports, commented code, hardcoded paths remove karo

Phase 3: Documentation (1 din)

■ **README.md**	Installation, quickstart, API reference, Docker setup
■ **CHANGELOG.md**	v0.1.0 release notes
■ **docstrings**	Sab public functions mein proper docstrings
■ **CONTRIBUTING.md**	How to contribute guide for open-source
■ **.github/ISSUE_TEMPLATE**	Bug report + feature request templates

Phase 4: Publish (1 din)

■ **PyPI account**	pypi.org pe account banao + API token generate karo
■ **Test on TestPyPI**	pehle test.pypi.org pe try karo
■ **pip install test**	Fresh venv mein pip install faceattend test karo

■ **GitHub Release**	v0.1.0 tag + release notes publish karo
■ **Share karo**	Reddit r/MachineLearning, LinkedIn, Twitter pe announce karo

6. Challenges aur Solutions

Challenge	Solution
TensorFlow heavy dependency	Optional install: pip install faceattend[gpu] vs base install
Model weights download	DeepFace auto-download karta hai — koi action needed nahi
Email credentials security	.env file use karo (already python-dotenv hai) — never hardcode
Hardcoded file paths	Config class banana ya os.getenv() use karo throughout
Large .pkl encoding file	.gitignore mein rakhna, user training run kare
Python version compatibility	Python 3.9+ target karo (TF requirement)
Docs heavy codebase	Sphinx + Read the Docs use karo for auto-generated docs

7. Tech Stack — Complete Reference

7.1 Core Libraries

Library	Version	Purpose
deepface	0.0.93	Face detection + recognition (Facenet model)
tensorflow	2.18.0	Backend for DeepFace models
opencv-python	4.11.0	Image processing, grid splitting
scikit-learn	1.6.1	Cosine similarity calculation
Flask	3.1.0	REST API server
flask-cors	5.0.1	CORS for Next.js frontend
pandas	2.2.3	CSV data handling
numpy	2.0.2	Numerical operations, embeddings
python-dotenv	latest	Environment variable management
reportlab	4.4.0	PDF generation (research paper + this doc!)

mtcnn	1.0.0	Multi-task CNN face detector
retina-face	0.0.17	Alternative face detector

7.2 Frontend (Next.js)

Next.js	React framework for the teacher dashboard
Tailwind CSS	Utility-first styling
shadcn/ui	Component library (components.json present)
TypeScript	Type-safe frontend code

7.3 Infrastructure

Docker	Both model (Python) and client (Node) containers
docker-compose	Single command to run full stack: docker-compose up
gunicorn	Production WSGI server for Flask

8. References

8.1 Core Papers & Libraries

- • **FaceNet** — Schroff et al. (2015). "FaceNet: A Unified Embedding for Face Recognition and Clustering." CVPR 2015. https://arxiv.org/abs/1503.03832
- • **DeepFace Library** — Serengil & Ozpinar. "LightFace: A Hybrid Deep Face Recognition Framework." ASYU 2020. https://github.com/serengil/deepface
- • **MTCNN** — Zhang et al. (2016). "Joint Face Detection and Alignment using Multi-task Cascaded CNNs." IEEE Signal Processing Letters. https://arxiv.org/abs/1604.02878
- • **RetinaFace** — Deng et al. (2019). "RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild." CVPR 2020. https://arxiv.org/abs/1905.00641
- • **ArcFace** — Deng et al. (2018). "ArcFace: Additive Angular Margin Loss for Deep Face Recognition." https://arxiv.org/abs/1801.07698

8.2 Python Packaging References

- • **Python Packaging Guide** — Official: https://packaging.python.org/en/latest/
- • **pyproject.toml spec** — PEP 517/518/621: https://peps.python.org/pep-0621/
- • **PyPI** — Python Package Index: https://pypi.org
- • **TestPyPI** — Test your package first: https://test.pypi.org
- • **GitHub Actions + PyPI publish** — https://docs.github.com/en/actions/use-cases-and-examples/publishing-packages/publishing-python-packages
- • **setuptools docs** — https://setuptools.pypa.io/en/latest/
- • **Semantic Versioning** — https://semver.org

8.3 Open-Source Best Practices

- • **Choose a License** — https://choosealicense.com (MIT already chosen ■)
- • **Keep a Changelog** — https://keepachangelog.com
- • **Contributor Covenant** — Code of conduct: https://www.contributor-covenant.org
- • **GitHub Actions CI/CD** — https://docs.github.com/en/actions
- • **Read the Docs** — Free documentation hosting: https://readthedocs.org
- • **Shields.io** — README badges (PyPI version, downloads, license): https://shields.io

8.4 Face Recognition Benchmarks (for Research Section)

- • **LFW (Labeled Faces in the Wild)** — Standard benchmark: http://vis-www.cs.umass.edu/lfw/
- • **MS-Celeb-1M** — Large-scale face dataset used for Facenet training
- • **VGGFace2** — https://github.com/ox-vgg/vgg_face2
- • **IJB-C** — IARPA Janus Benchmark-C for face recognition at scale

8.5 Useful Tools for Your Project

- • **Twine** — Upload to PyPI: pip install twine
- • **Build** — Build wheel: pip install build → python -m build
- • **Bumpversion** — Auto-increment version: pip install bump2version
- • **pre-commit** — Code quality hooks: https://pre-commit.com
- • **Black** — Python formatter: pip install black
- • **Ruff** — Fast Python linter: pip install ruff
- • **pytest** — Testing framework: pip install pytest
- • **Sphinx** — Auto-generate docs from docstrings: pip install sphinx

9. Quick Commands — Yaad Rakhne Waale

Run the full stack locally

```
# Docker se (easiest)
docker-compose up

# Manually
cd model && python api/app.py # Backend on :8000
cd client && npm run dev # Frontend on :3000
```

Train face encodings

```
cd model
python deepface_model/main.py # trains + saves face_encodings.pkl
```

Build Python package (after pyproject.toml is ready)

```
pip install build twine
python -m build # creates dist/*.whl and dist/*.tar.gz
twine upload --repository testpypi dist/* # test first
twine upload dist/* # publish to PyPI
```

API Endpoints

POST /upload	Upload classroom image(s), get recognized faces JSON back
GET /	Health check — returns welcome message

■ ***Reminder:*** *Yeh document living notes hai — jab bhi naya feature add karo ya koi decision lo, isko update karo taaki future mein confusion na ho.*
■ ***Next immediate step:*** *pyproject.toml banana aur faceattend/ package directory create karna — baaki sab uske baad aata hai.*

— Generated by GitHub Copilot Agent on 03 April 2026